

Space Colony — Final Delivery

Nicolás Acero Ladino, Erik Fernandez Rios, Laura Paez Cifuentes

Computer Sciences I

Universidad Distrital Francisco José de Caldas — Week 16

Agenda

- 1 Game Overview
- 2 Algorithms
- 3 Data Structures
- 4 System Architecture
- 5 Results and Conclusions

Game Overview

- Competitive grid strategy game on an 8×8 board (Player vs. AI)
- **Expansion Phase:** colonise empty tiles using a limited energy budget
- **Conquest Phase:** attack adjacent enemy tiles to dominate the map
- Objective: control the largest territory when the grid is full

$$Power = Resources + Random(1,6)$$



Game Phases

- **Expansion:** Players alternate turns colonising adjacent empty tiles according to their available energy
- **Conquest:** Triggered once every tile is occupied. Players attack adjacent enemy tiles
- The AI uses **Greedy** during expansion and **Backtracking** to plan colonisation routes



Greedy Algorithm — Definition

What is a Greedy Algorithm?

At every step it picks the **locally optimal** choice without considering future consequences. It is fast and simple, though it does not always guarantee a global optimum.

Key properties:

- Greedy choice: local maximum or minimum
- Single pass through candidates: $O(n)$
- Never backtracks or explores alternatives

Complexity in this project:

```
GREEDY_SELECT_MOVE(available_cells, energy):
```

```
best_cell ← null  
max_value ← -∞
```

```
for each cell in available_cells:  
    if cell.energy_cost ≤ energy:  
        value ← cell.resources
```

```
    if value > max_value:  
        max_value ← value  
        best_cell ← cell
```

```
return best_cell
```

```
GREEDY_ATTACK(neighbor_enemy_cells):
```

```
target ← null  
lowest_defense ← +∞
```

```
for each cell in neighbor_enemy_cells:  
    defense ← cell.resources
```

```
    if defense < lowest_defense:  
        lowest_defense ← defense  
        target ← cell
```

Greedy Algorithm in Space Colony

Expansion: the AI selects the affordable adjacent empty tile with the highest resource value.

```
1 best_cell = None
2 max_value = -inf
3 for cell in available_cells:
4     if cell["energy_cost"] <=
5         energy:
6         if cell["resources"] >
7             max_value:
8             max_value = cell["
9                 resources"]
10             best_cell = cell
11 return best_cell
```

Conquest: the AI attacks the adjacent enemy tile with the *lowest* resource value, maximising capture probability.



Backtracking Algorithm — Definition

What is Backtracking?

Explores **all possible combinations** of decisions, undoing invalid moves and reversing when a constraint is violated. With proper pruning it becomes practical for moderate search spaces.

```
BACKTRACK(current_position, visited_cells, total_resources, energy):
```

```
    if no more possible moves:  
        update best_solution  
        return
```

```
    for each neighbor of current_position:
```

```
        if neighbor is not visited  
        and satisfies_distance(neighbor)  
        and neighbor.energy_cost ≤ energy:
```

```
            mark neighbor as visited
```

```
            BACKTRACK(  
                neighbor,  
                visited_cells,  
                total_resources + neighbor.resources,  
                energy - neighbor.energy_cost  
            )
```

```
            unmark neighbor
```

```
    satisfies_distance(position):
```

```
        for each colony in existing_colonies:  
            if distance(position, colony) < minimum_distance:  
                return false
```

```
    return true
```

Backtracking in Space Colony

The AI plans the **best colonisation sequence** satisfying:

- Minimum Manhattan distance between same-owner colonies (≥ 2)
- Available energy budget
- Adjacency to the current frontier

```
1 BACKTRACK(pos, visited, res,
2         energy):
3     if no valid moves:
4         update best_solution(res)
5         return
6     for neighbor of pos:
7         if valid(neighbor, energy):
8             mark neighbor
9             BACKTRACK(neighbor,
10                      visited, res+neighbor.res
11                      ,
12                      energy - cost)
13             unmark neighbor #
```

Pruning strategies applied:

- ① Candidates sorted **descending by resources** so high-value paths are explored first, enabling earlier cutoffs

Only the **first cell** of the best sequence is executed per turn.

AVL Tree — Definition

What is an AVL Tree?

A **self-balancing** binary search tree. After every insertion or deletion it checks the *balance factor* and applies rotations to maintain minimum height.

Why AVL and not BST?

- AVL always guarantees $O(\log n)$ height

Operation	BST worst case	AVL guaranteed
Insert	$O(n)$	$O(\log n)$
Search	$O(n)$	$O(\log n)$
Delete	$O(n)$	$O(\log n)$

AVL Tree in Space Colony

Role: stores the *Resource Deposit Tree* — all colonies ordered by resource value.

Operations used by the AI:

- `find_max_by_owner` → richest own colony (greedy reference)
- `find_min_by_owner` → weakest own colony
- `insert` / `remove` on every colonisation or capture event

Balance rotations (C++):

```
1 // Left-left case
2 if (balance > 1 &&
3     get_balance(node->left) >=
4         0)
5     return rotate_right(node);
6
7 // Left-right case
8 if (balance > 1 &&
9     get_balance(node->left) <
10        0) {
11     node->left =
12         rotate_left(node->left);
13     return rotate_right(node);
14 }
```

What is a Singly Linked List?

A linear structure where each node stores a **value** and a **pointer** to the next node. No contiguous memory is required; it grows and shrinks in constant time at the tail.

Complexity:

Operation	Complexity
append (insert at tail)	$O(1)$
remove_last	$O(n)$
print_all / traversal	$O(n)$

Implementation: raw pointers
new/delete — no `std::list` or STL containers.

Linked List in Space Colony

Role: stores the *Settlement History List* — a chronological log of every colonisation event during the match.

Each node holds:

- Position (x, y) of the colonised tile
- owner — player or AI
- resources — value at the moment of colonisation
- next — pointer to the next event

The list is traversed to reconstruct history and generate the `state.json` file that Python reads to render the board.

```
1 void LinkedList::append(  
2     int x, int y,  
3     const string& owner,  
4     int resources) {  
5     Node* n      = new Node();  
6     n->x         = x;  
7     n->y         = y;  
8     n->owner      = owner;  
9     n->resources  = resources;  
10    n->next       = nullptr;  
11    if (tail == nullptr) {  
12        head = tail = n;  
13    } else {  
14        tail->next = n;  
15        tail = n;  
16    }  
17    ++count;  
18 }
```

- **Python (Pygame):** algorithm logic, grid UI, HUD, and animations
- **C++ Engine:** Linked List + AVL Tree, game-state processing
- **JSON Bridge:** exclusive communication through plain files — no sockets, no bindings

Data flow:

`main.py` $\xrightarrow{\text{input.json}}$ `main.cpp` $\xrightarrow{\text{state.json}}$ `main.py`

JSON Bridge — Data Contract

input.json (Python → C++):

```
1 {
2   "action": "colonize",
3   "x": 3,
4   "y": 4,
5   "owner": "player",
6   "energy_cost": 5
7 }
```

state.json (C++ → Python):

```
1 {
2   "grid": [[...], ...],
3   "owner_grid": [[...], ...],
4   "colonies": [
5     {"x": 3, "y": 4,
6      "owner": "player",
7      "resources": 50}
8   ],
9   "ai_energy": 80,
10  "player_energy": 75,
11  "phase": "expansion"
12 }
```

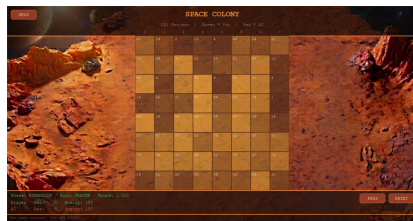
- Integration done **exclusively** through file I/O
- `bridge.py` reads `state.json` and writes `input.json` every turn
- The C++ engine processes the action, updates the list and tree, and writes the new state
- Plain-text format makes debugging straightforward — open the file and read the state

Work Distribution

Role	Language	Responsibilities
Engine Developer	C++	Linked List, AVL Tree, main.cpp: reads input.json and writes state.json
Algorithm Developer	Python	greedy.py, backtracking.py, inte- gration with bridge.py
UI Developer	Python / Pygame	8×8 grid, HUD, ani- mations in main.py, bridge.py file I/O

Results

- **Efficient expansion:** the greedy algorithm always selects the highest-value tile in $O(n)$
- **Valid placement:** backtracking guarantees no colony violates the minimum distance constraint
- **Fast lookups:** the AVL Tree keeps `find_max` at $O(\log n)$ regardless of insertion order
- **Complete history:** the Linked List records every colonisation event without data loss
- Matches are genuinely competitive — either side can win



Conclusions

- Algorithms: greedy and backtracking ain't they were very important in the Real-time AI decision in game.
- *Data structures: the AVL tree and Linked List, implemented from scratch in C++, provides efficiency and Game state management.
- Integration: JSON bridge demonstrates that Systems written in different languages can cooperate by taking the best of each other.
- This work allowed a practical execution of the topics seen throughout the semester, being of great help and benefiting our knowledge.

THANK YOU

Computer Sciences I — Universidad Distrital Francisco José de Caldas